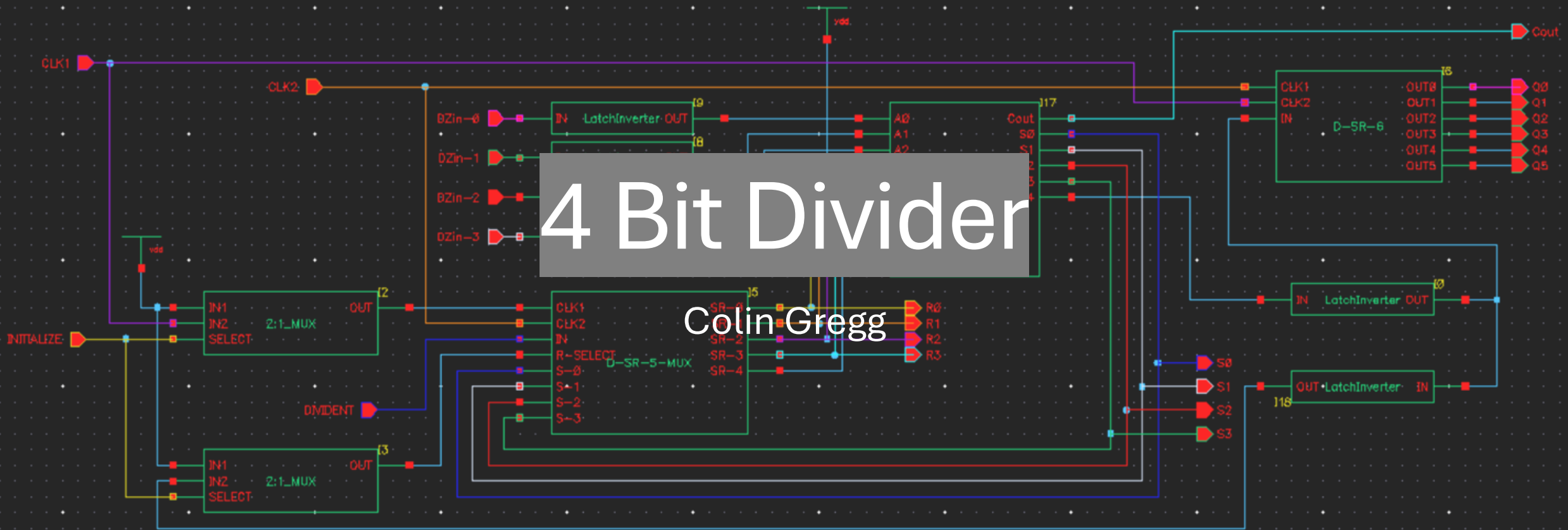


4 Bit Divider

Colin Gregg



Specifications

- 6-bit (unsigned) dividend.
- 4-bit (unsigned) divisor.
- Divisor > 0 .
- Divisor $<$ Dividend.
- Outputs are the 6-bit quotient and 4-bit remainder.
- Quotient should be loaded into a shift register.

Division Algorithm

- I used the same algorithm prescribed to us with a slight modification to fit the 6-bit dividend.
- The method is identical to long division by hand.
- Involves repeated subtraction of divisor from partial sums of the dividend.
- After each subtraction, shift the dividend 1 bit to the left.
- If the divisor is smaller than the partial sum of the dividend, then $C_{out} = 1$ and $S_4 = 0$ and the three most significant bits of the partial sum must be replaced by the first three bits of the previous result of the subtraction.
- The process is repeated for 6 shifts (+ a dummy shift).
- The quotient is read as the C_{out} bits, with the first being the MSB, and last the LSB.
- If the final C_{out} is 1, then the remainder is the result of the final subtraction, otherwise it is the partial sum (from the final subtraction) of the dividend.

Example... 45/9

```

101101
/
001001
-----
000101
R0000
    
```

Partial Sum of dividend

Shift dividend bit to
create new partial sum

Partial sum > divisor,
Output 1 for quotient

Replace partial sum of
dividend with first 3 bits of
result of subtraction

	Carry Out	S4	2's complement										
	Cout												
DUMMY SHIFT		0	0	0	0	0	1	0	1	1	0	1	
		1	0	1	1	1							
	0	1	0	1	1	1							
SHIFT 1		0	0	0	0	1	0	1	1	0	1	x	
		1	0	1	1	1							
	0	1	1	0	0	0							
SHIFT 2		0	0	0	1	0	1	1	0	1	x	x	
		1	0	1	1	1							
	0	1	1	0	0	1							
SHIFT 3		0	0	1	0	1	1	0	1	x	x	x	
		1	0	1	1	1							
	0	1	1	1	0	0							
SHIFT 4		0	1	0	1	1	0	1	x	x	x	x	
		1	0	1	1	1							
	1	0	0	0	1	0							
SHIFT 5		0	0	1	0	0	1	x	x	x	x	x	
		1	0	1	1	1							
	0	1	1	0	1	1							
SHIFT 6		0	1	0	0	1	x	x	x	x	x	x	
		1	0	1	1	1							
	1	0	0	0	0	0							
QUOTIENT:							000101			i.e.	45/9 = 5 R0		
REMAINDER:							0000						

Example... 45/9

$$\begin{array}{r} 101101 \\ / \\ 001001 \end{array}$$

000101
R0000

Quotient

Remainder

LSB

	Cout	0	0	0	0	0	1	0	1	1	0	1
DUMMY SHIFT		0	0	0	0	0	1	0	1	1	0	1
		1	0	1	1	1						
SHIFT 1	0	0	0	0	0	1	0	1	1	0	1	x
		1	0	1	1	1						
	0	1	1	0	0	0						
SHIFT 2		0	0	0	1	0	1	1	0	1	x	x
		1	0	1	1	1						
	0	1	1	0	0	1						
SHIFT 3		0	0	1	0	1	1	0	1	x	x	x
		1	0	1	1	1						
	0	1	1	1	0	0						
SHIFT 4		0	1	0	1	1	0	1	x	x	x	x
		1	0	1	1	1						
	1	0	0	0	1	0						
SHIFT 5		0	0	1	0	0	1	x	x	x	x	x
		1	0	1	1	1						
	0	1	1	0	1	1						
SHIFT 6		0	1	0	0	1	x	x	x	x	x	x
		1	0	1	1	1						
	1	0	0	0	0	0						

QUOTIENT: 000101 i.e. 5
 REMAINDER: 0000 R0

Circuit Design

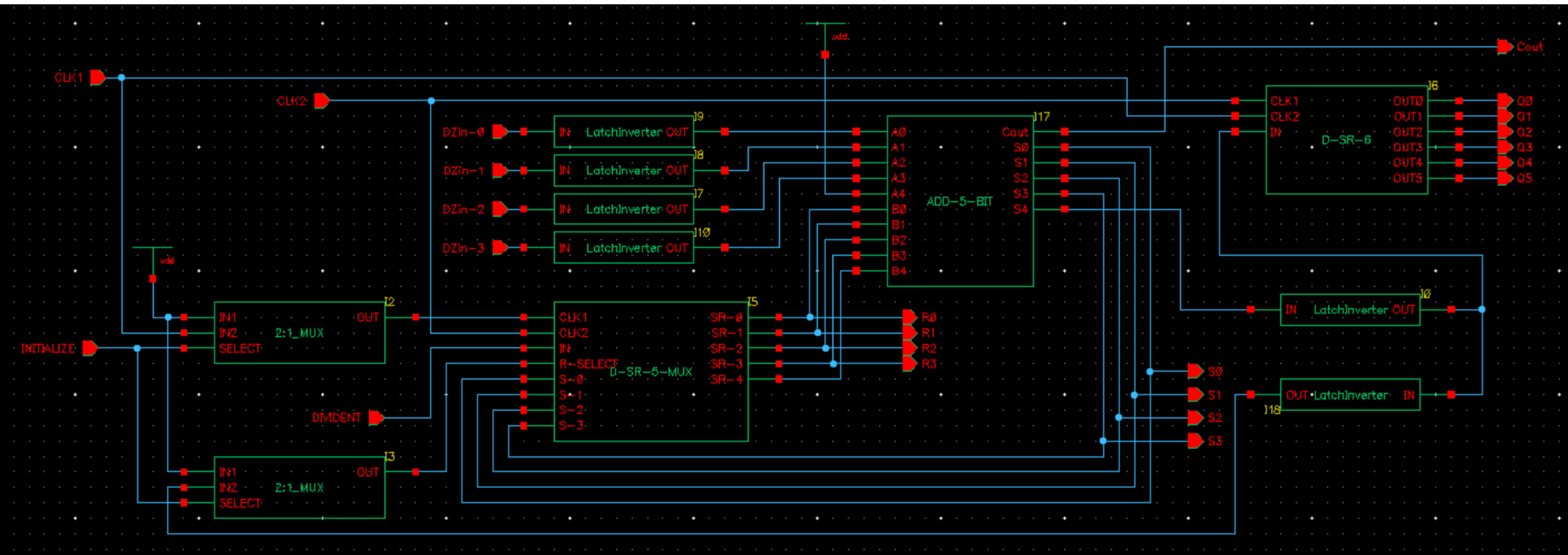
1

- I chose the “zero-initialize” setup for the circuit implementation.
- This means, besides executing the algorithm described just now, it will have an extra bit in front of every input which will initialize the registers with proper values so there will be no floating nodes or nodes at 1V.
- The dividend is fed serially into a 5-bit shift register, modified with 2:1 Muxes to replace the final four bits of the register with the previous result of the subtraction if S4 is low.
- Outputs of 5-bit SR fed into 5-bit adder hard wired to do 2's complement subtraction with the divisor.

Circuit Design...

2

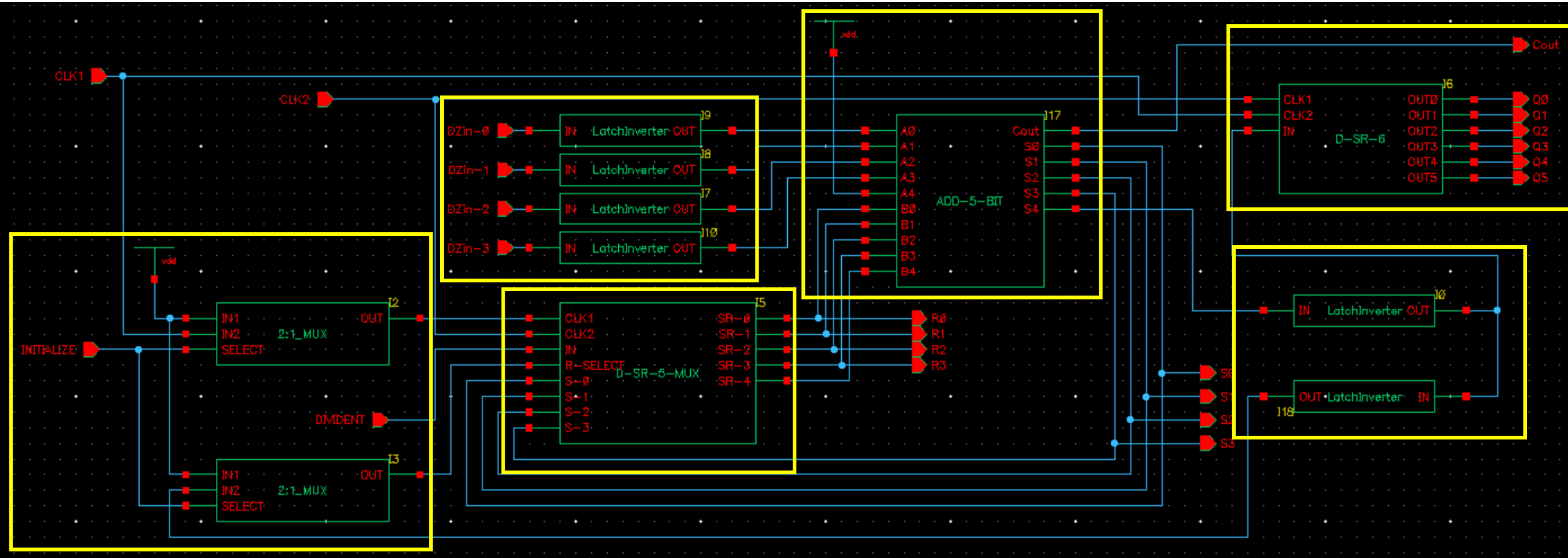
- This is my final circuit:



Circuit Design...

3

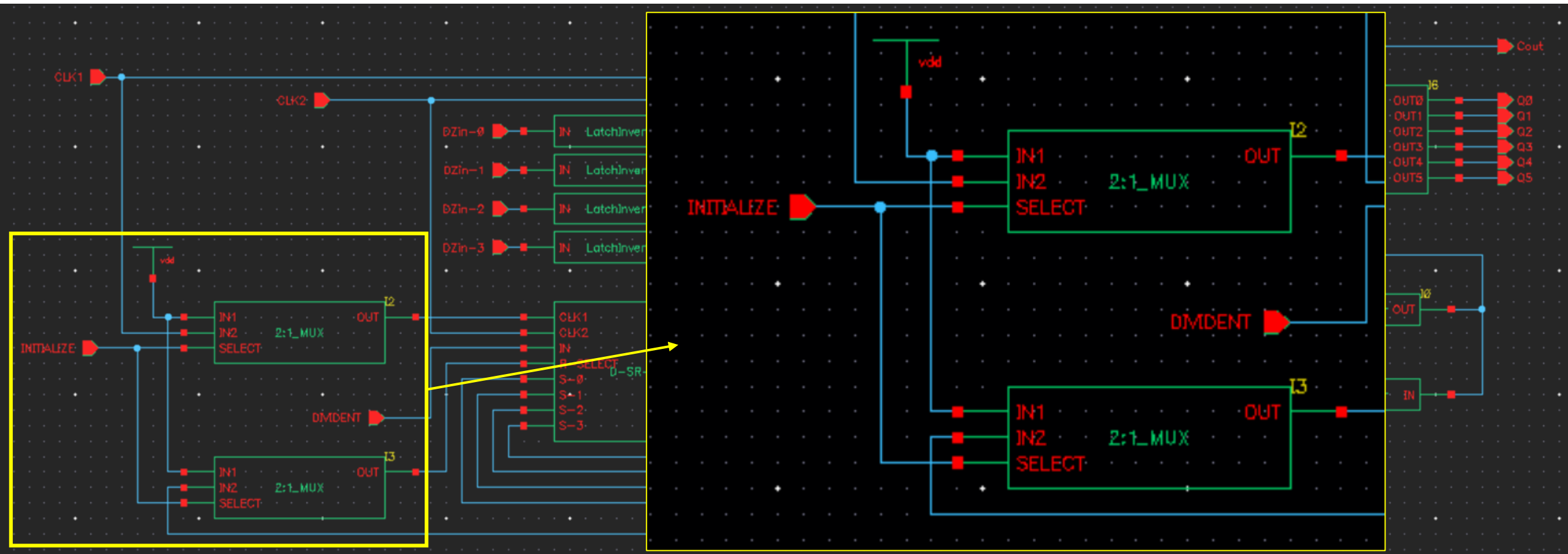
- And its various parts...



Circuit Design...

4

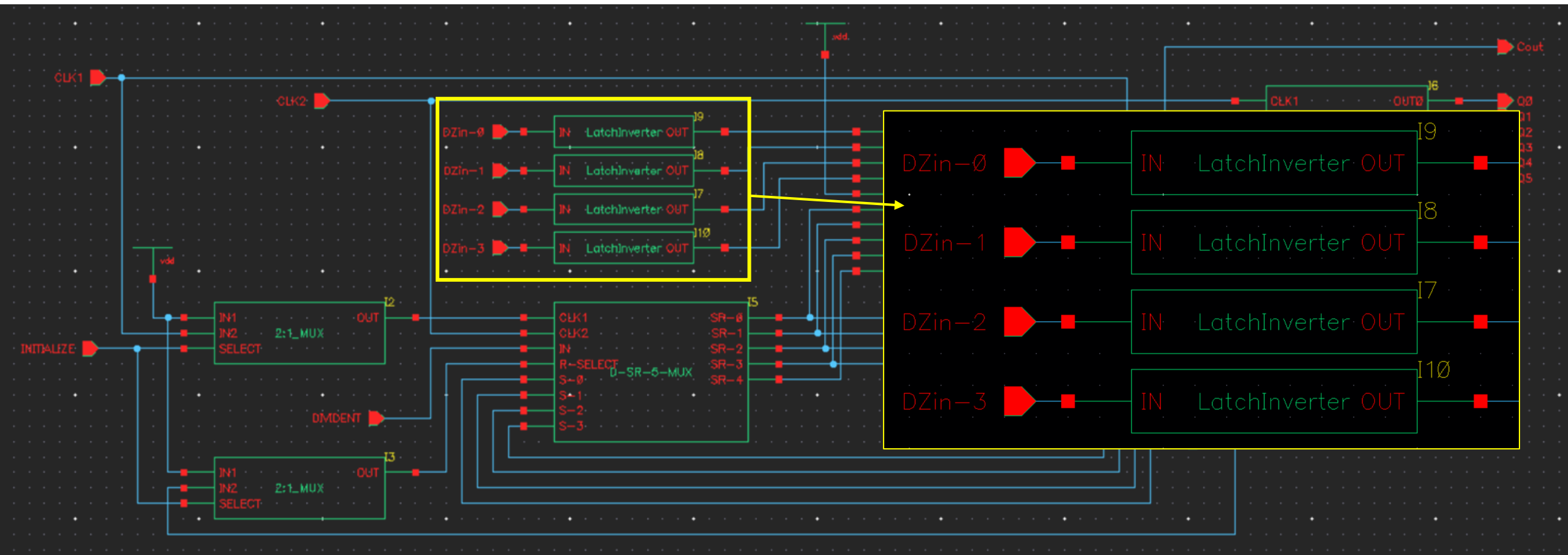
- Initializes registers to zero establishing the dummy shift...



Circuit Design...

5

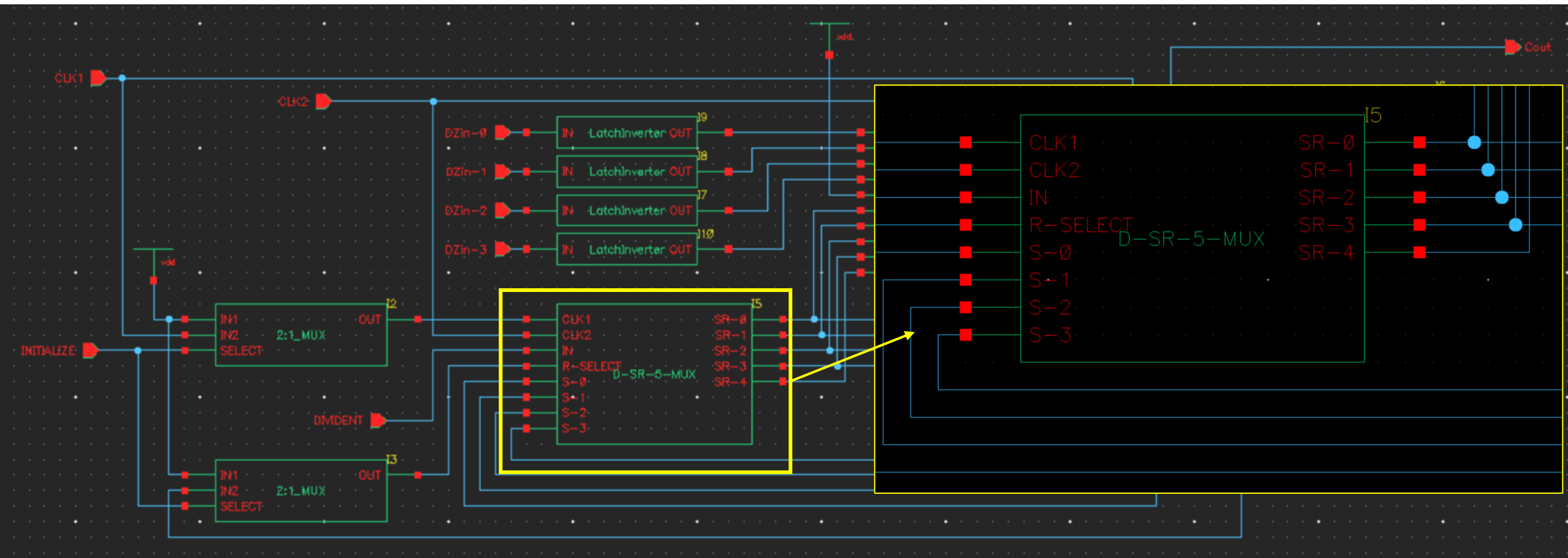
- Takes complement of divisor to do 2's complement subtraction...



Circuit Design...

6

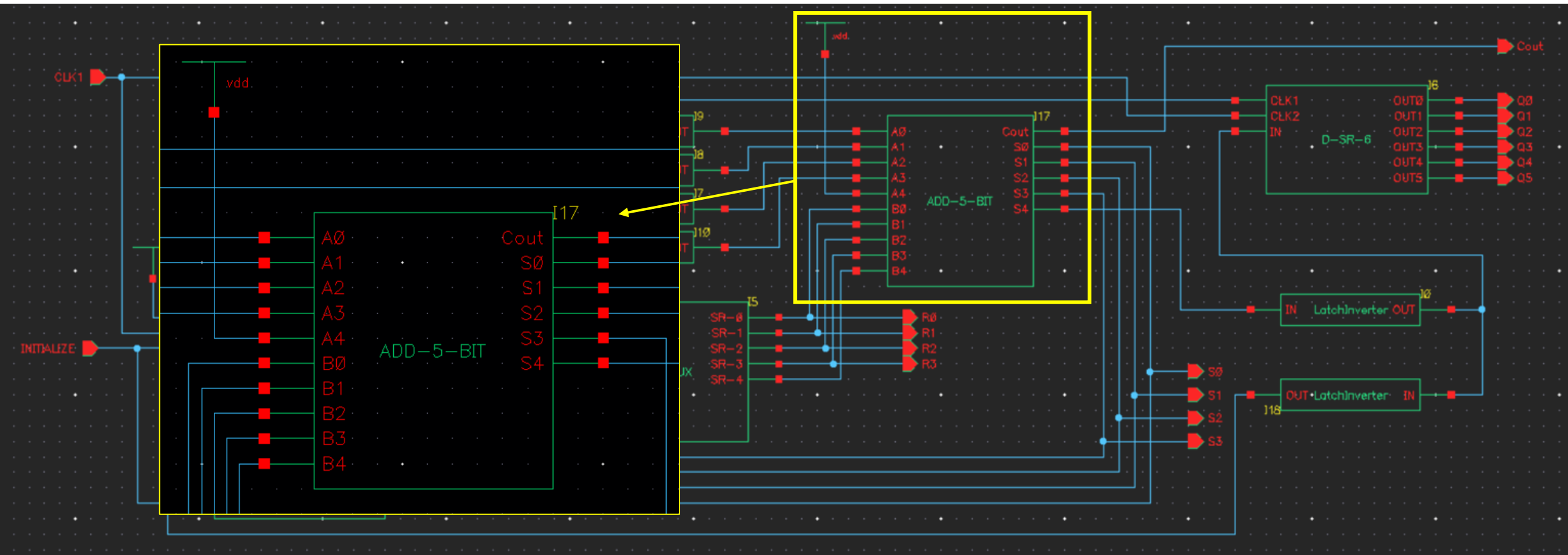
- Shifts bits to create each partial sum and does replacing mechanism



Circuit Design...

7

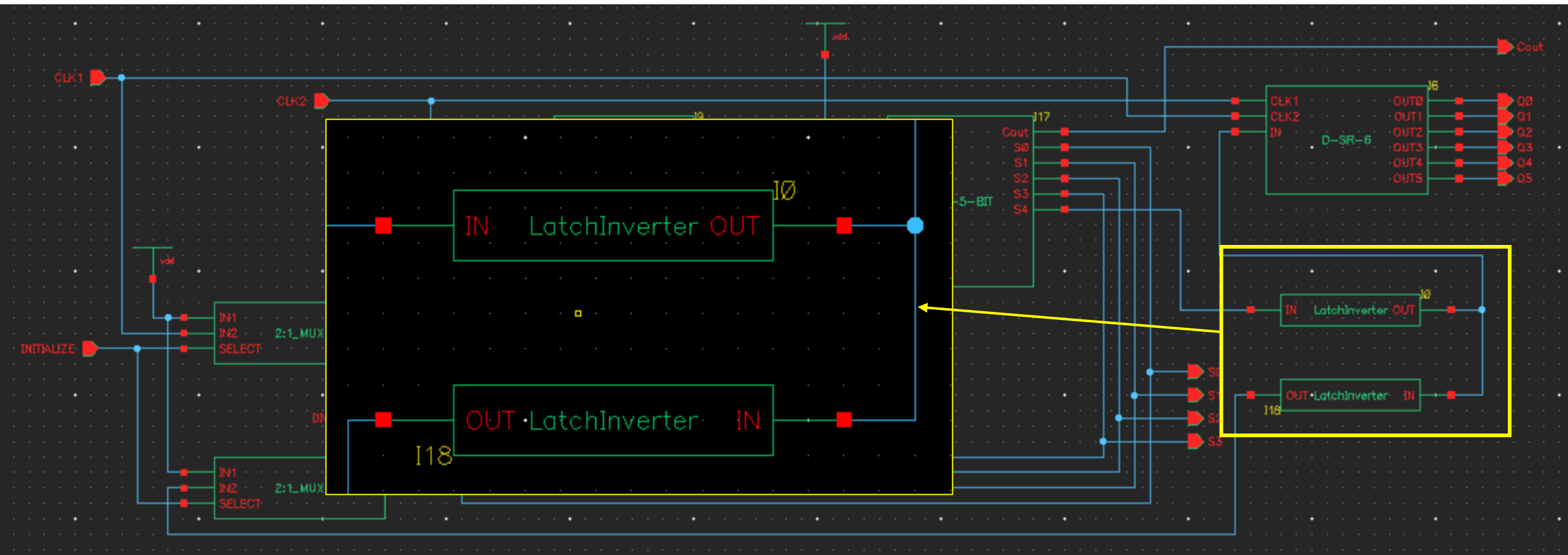
- Hardwired to do 2's complement subtraction of 5 bit numbers...



Circuit Design...

8

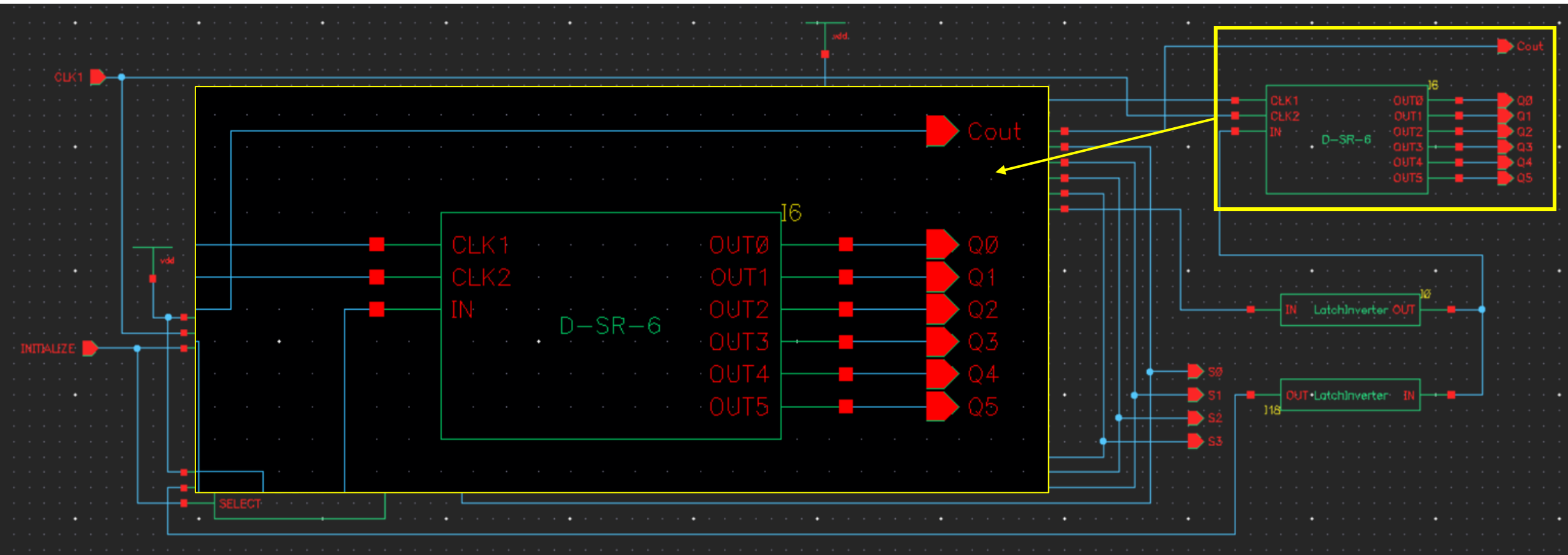
- Strengthens current of S4 output to drive replacing mechanism...



Circuit Design...

9

- 6-bit SR to display results (and Cout)



Circuit Components

1

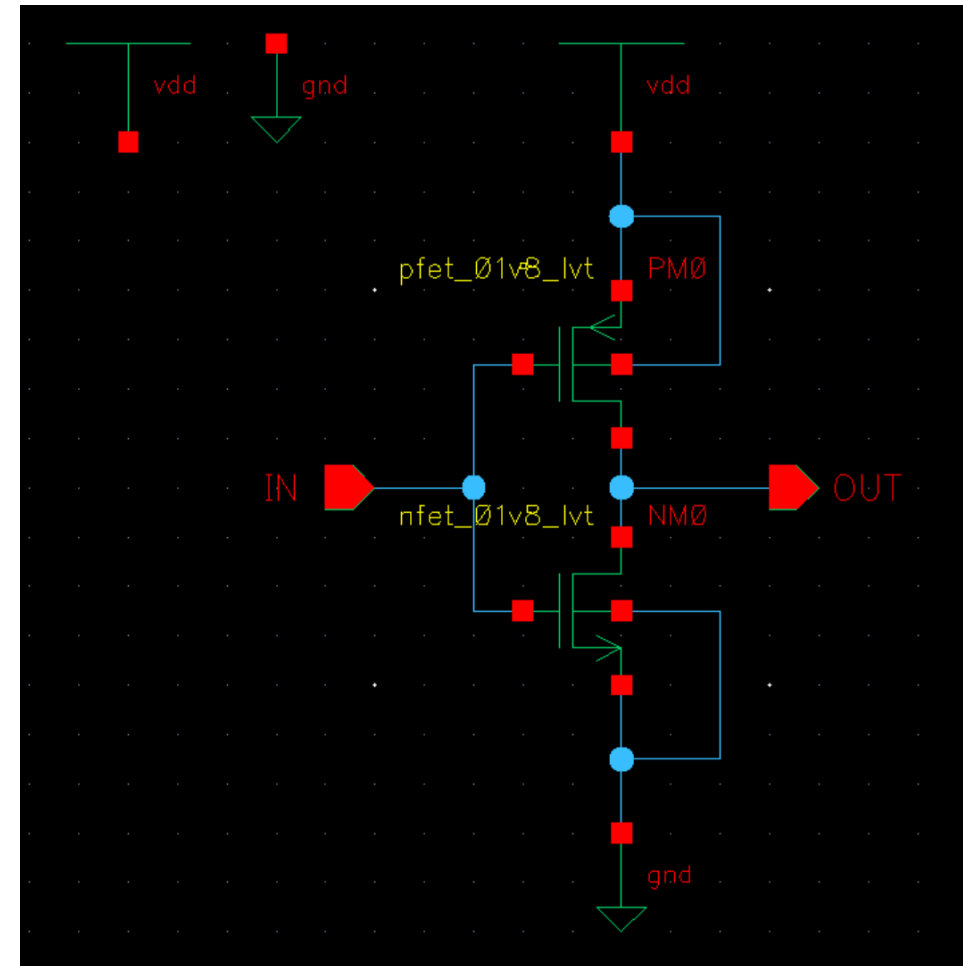
- The components I designed for this project are:
 - Inverter
 - Pass Gate
 - 2:1 MUX
 - 5-bit Shift Register (modified with 2:1 MUXES and initialize input)
 - 5-bit Carry-Ripple adder (hardwired for 2's complement subtraction)
 - 6-bit Shift Register

Circuit Components...

2

Inverter

- Used same sizing of transistors as previous project
- NMOS:
 - Length - 600nm
 - Width – 6um
- PMOS:
 - Length - 600nm
 - Width – 12um

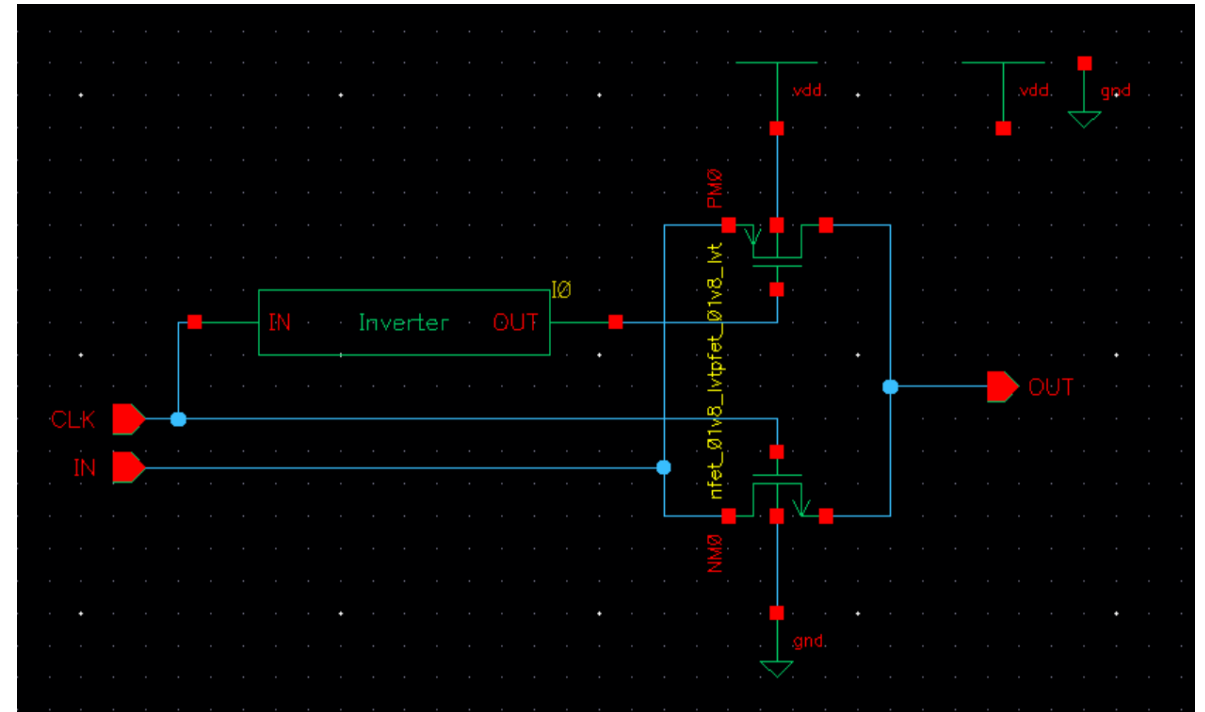


Circuit Components...

3

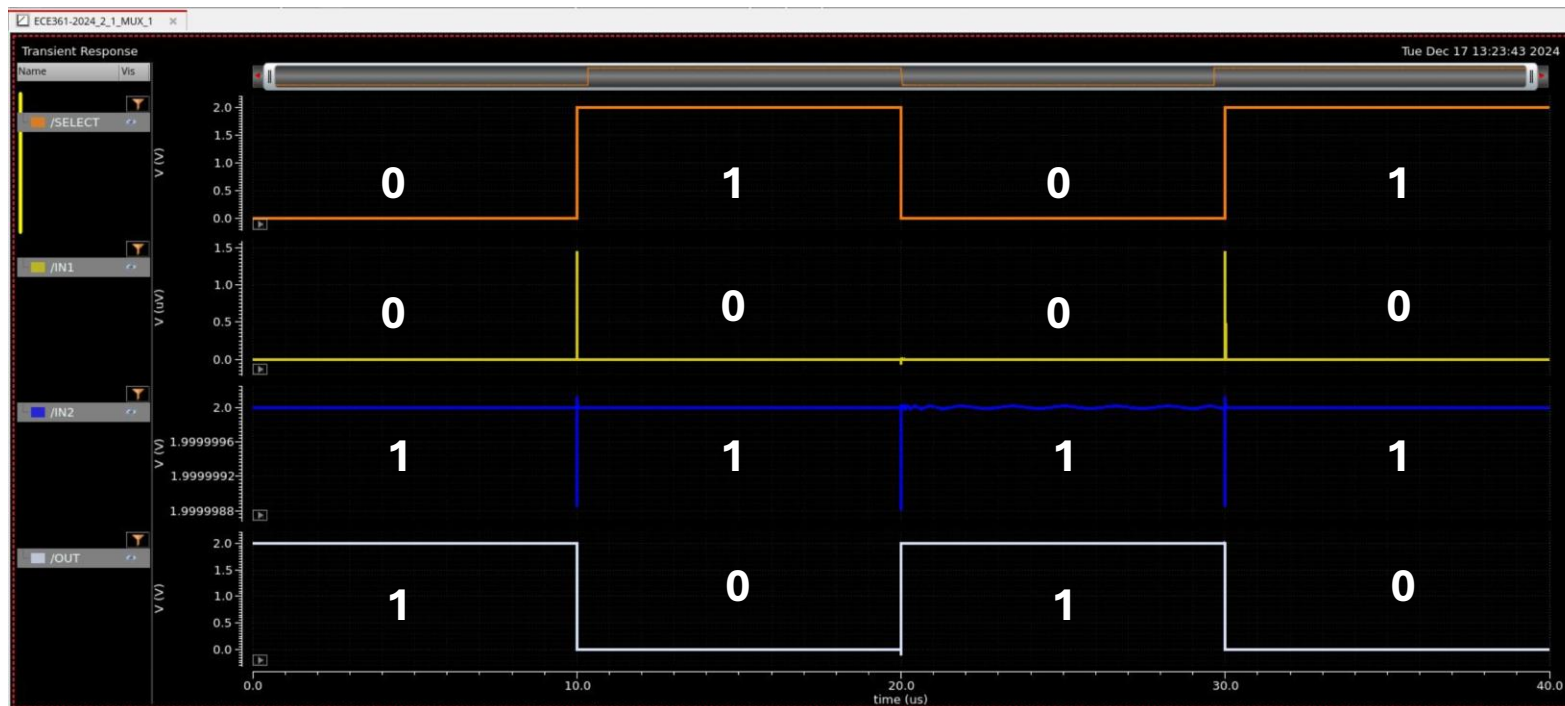
Pass Gate

- I changed my design from the previous project to:
 - Pass on high clock
 - Use physical inverter in design to get CLKBAR signal instead of adding a third input

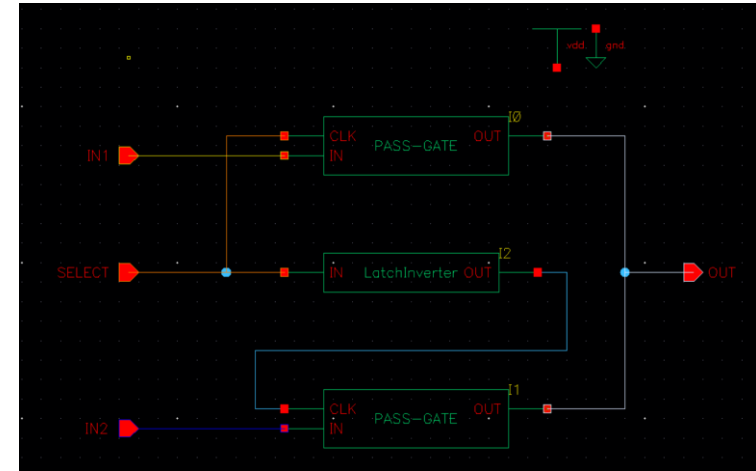


Circuit Components...

2:1 MUX



4



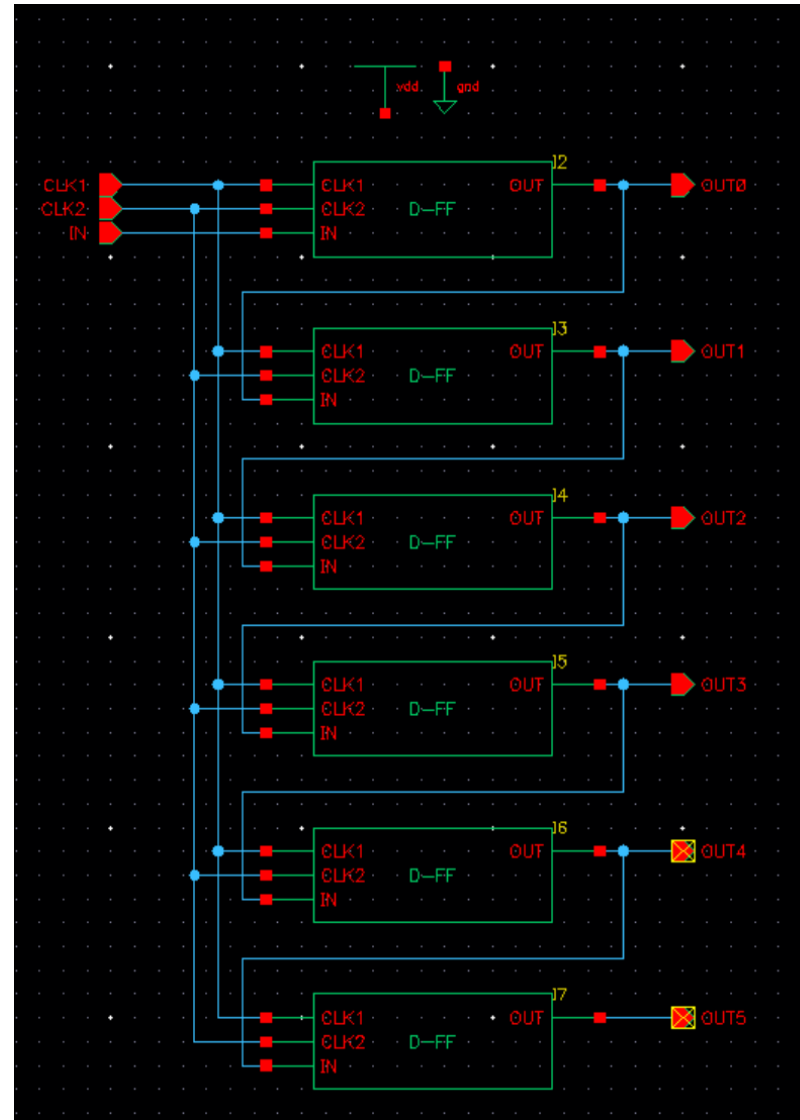
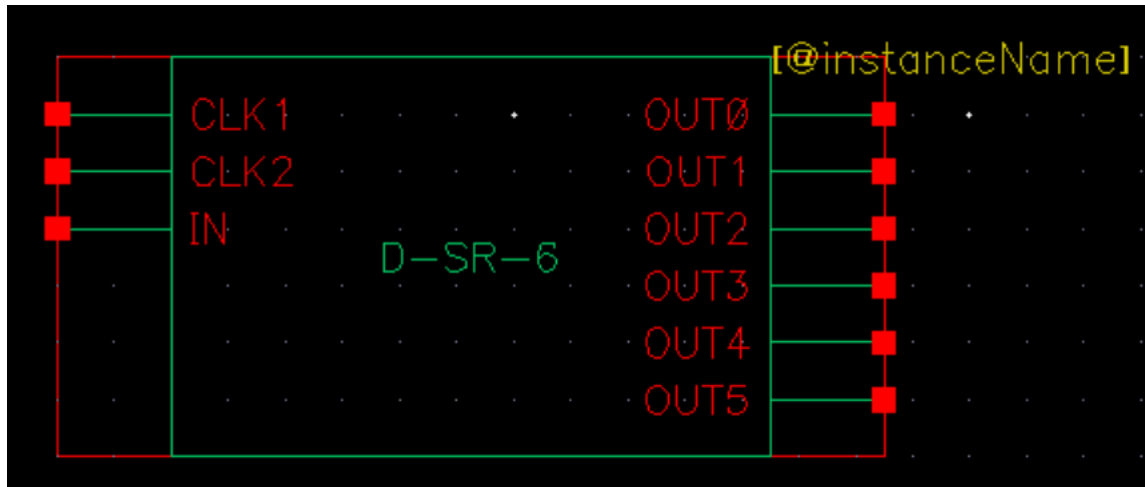
- In1 is always low, In2 is always high
- When select is low, chooses In2, when select is high, chooses In1

Circuit Components...

5

D-SR-6

- I remade the latches and flip flops needed for the 6-bit dynamic shift register
- Here is the schematic and cellview of the register



Circuit Components...

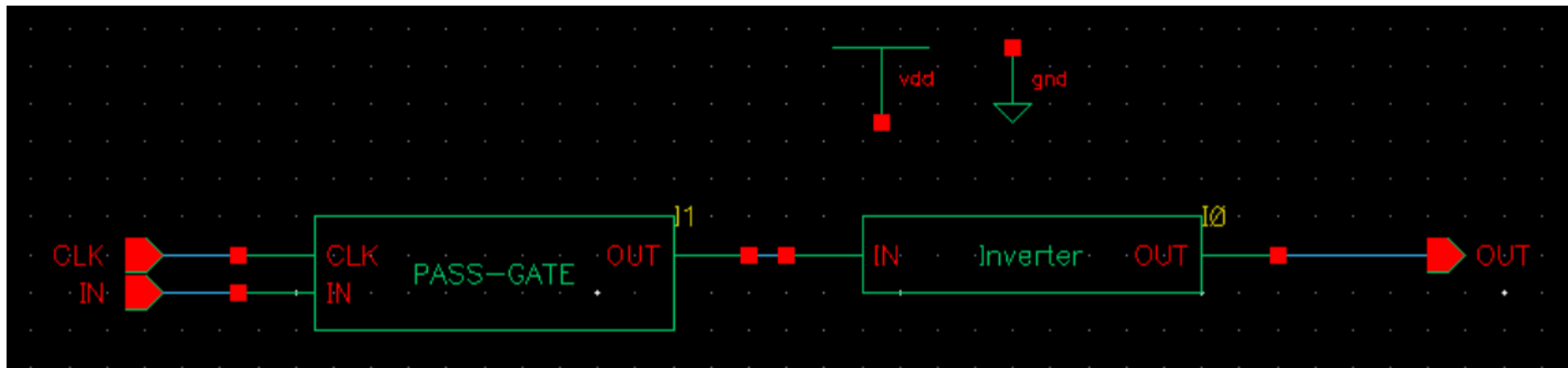
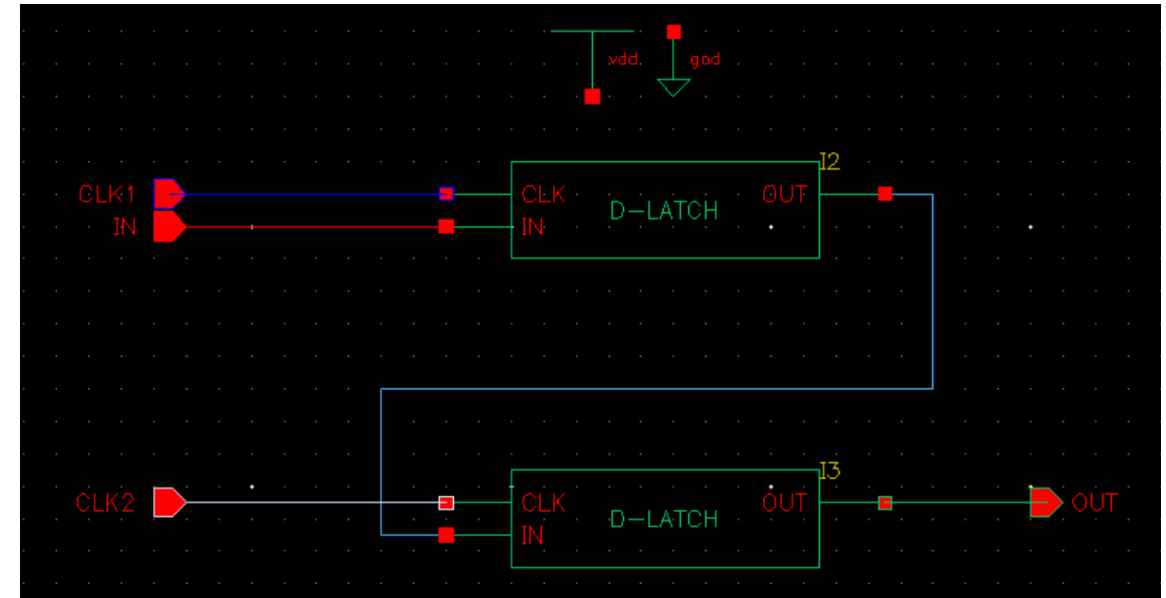
D-SR-6

- These are the latches used of the FF's and the FF's used for the register

D-Latch

D-FF

6

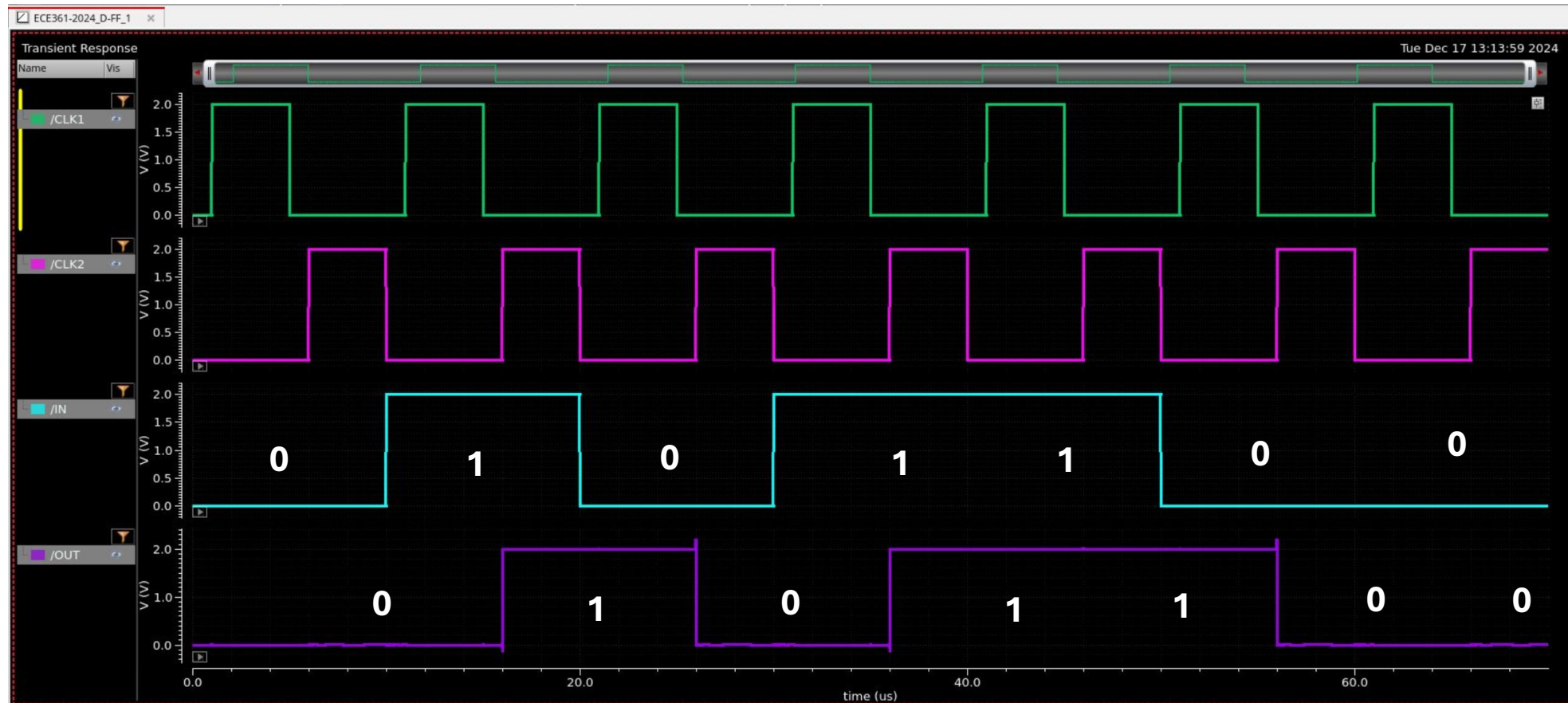


Circuit Components...

7

D-SR-6

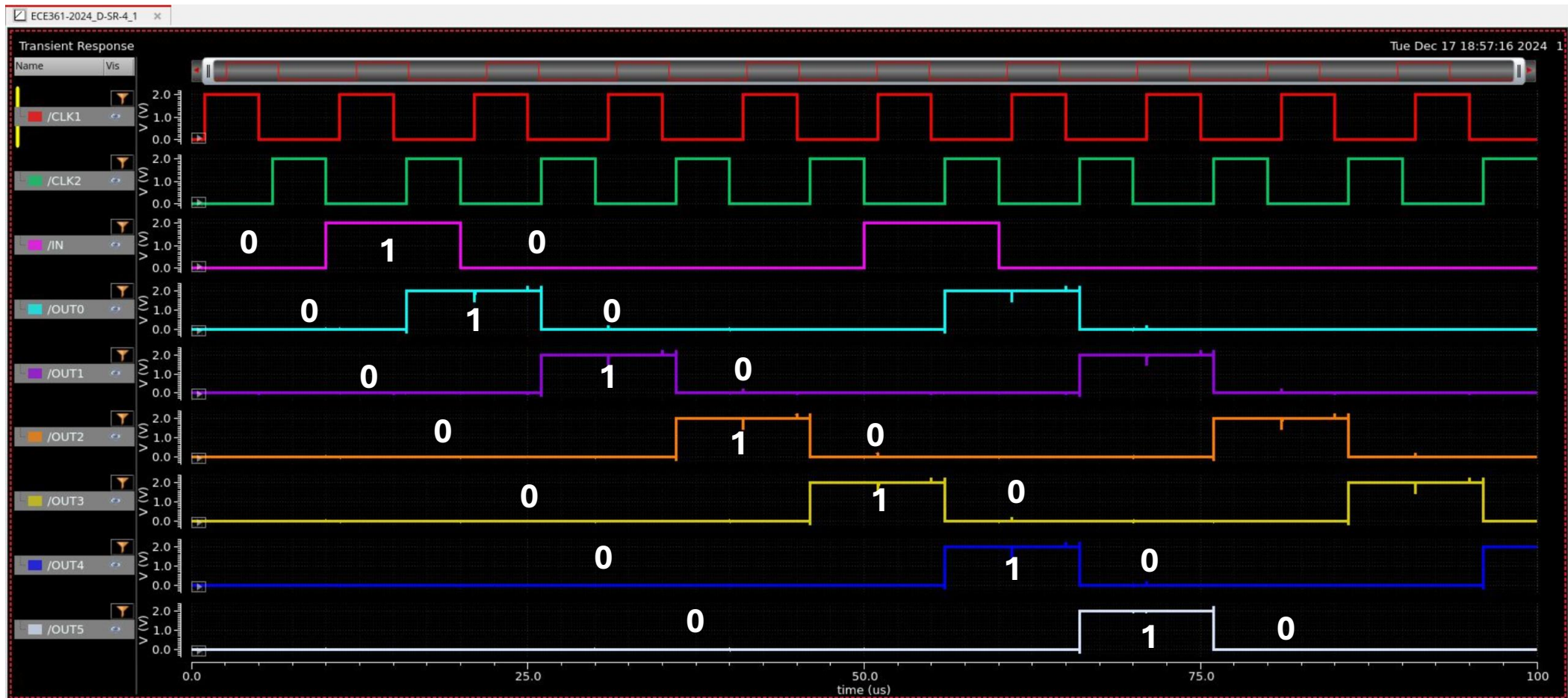
D-FF Simulation



Circuit Components...

8

D-SR-6: Simulation

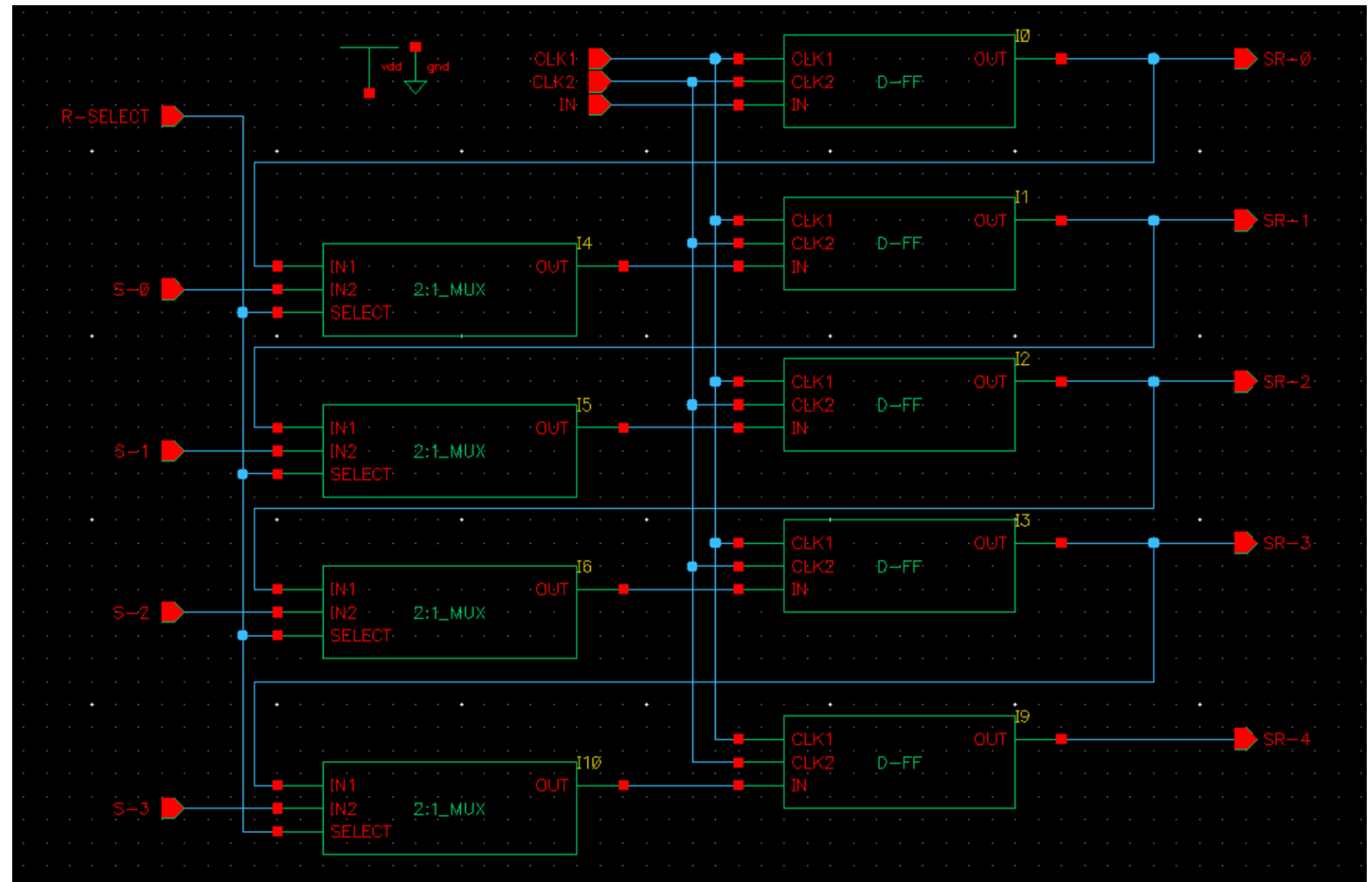


Circuit Components...

9

5-bit D-SR-MUX

- Similar to the SR made before.
- Uses 2:1 MUXES to replace the three most significant output bits if the R-SELECT input is low.
- Implements the replacing mechanism of division algorithm.



Circuit Components...

5-bit D-SR-MUX - Simulation

B: 0110 0110

S: x101 x101

Select: 1111 0000

10

After half-period
delay, outputs
according to S3-
S0

When
goes low,
switch

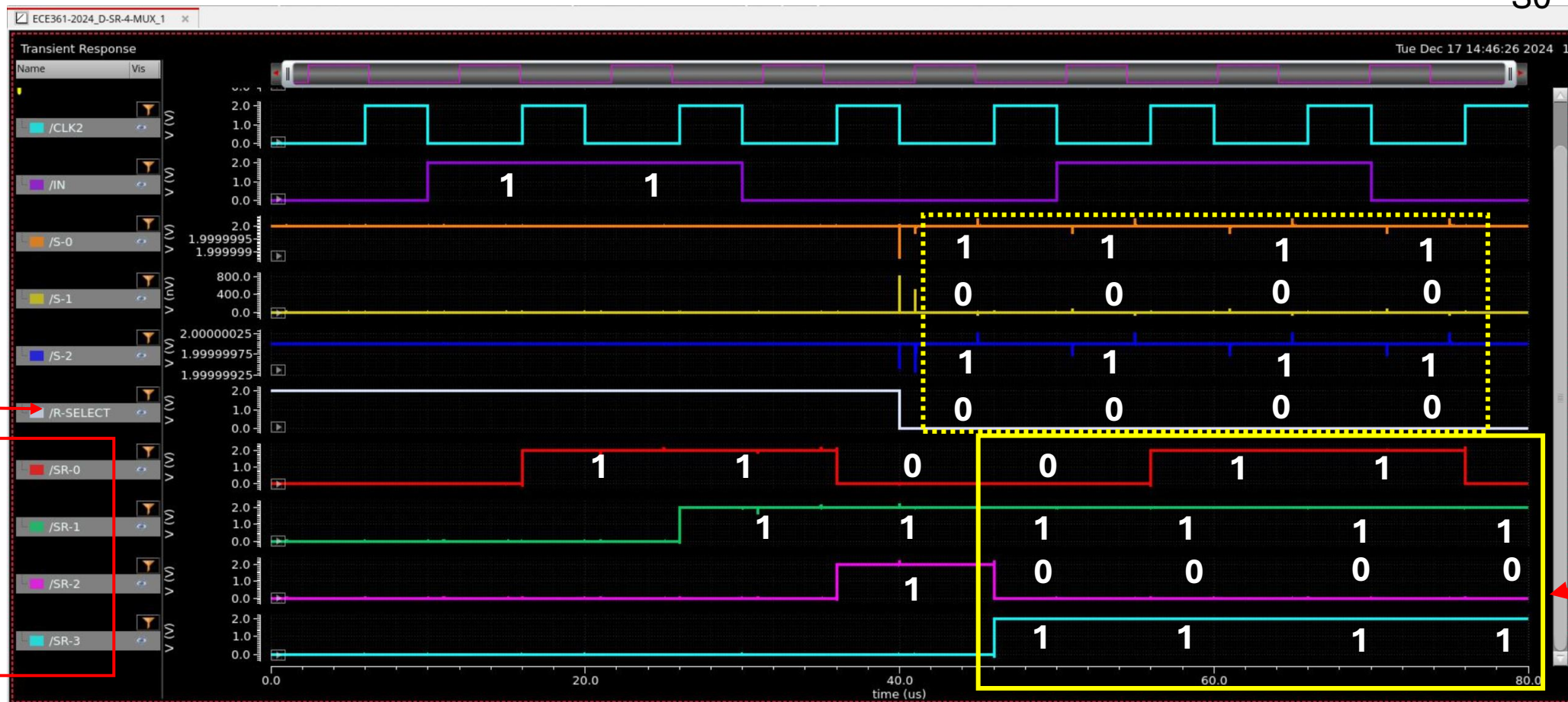
Outputs:

SR-0

SR-1

SR-2

SR-3

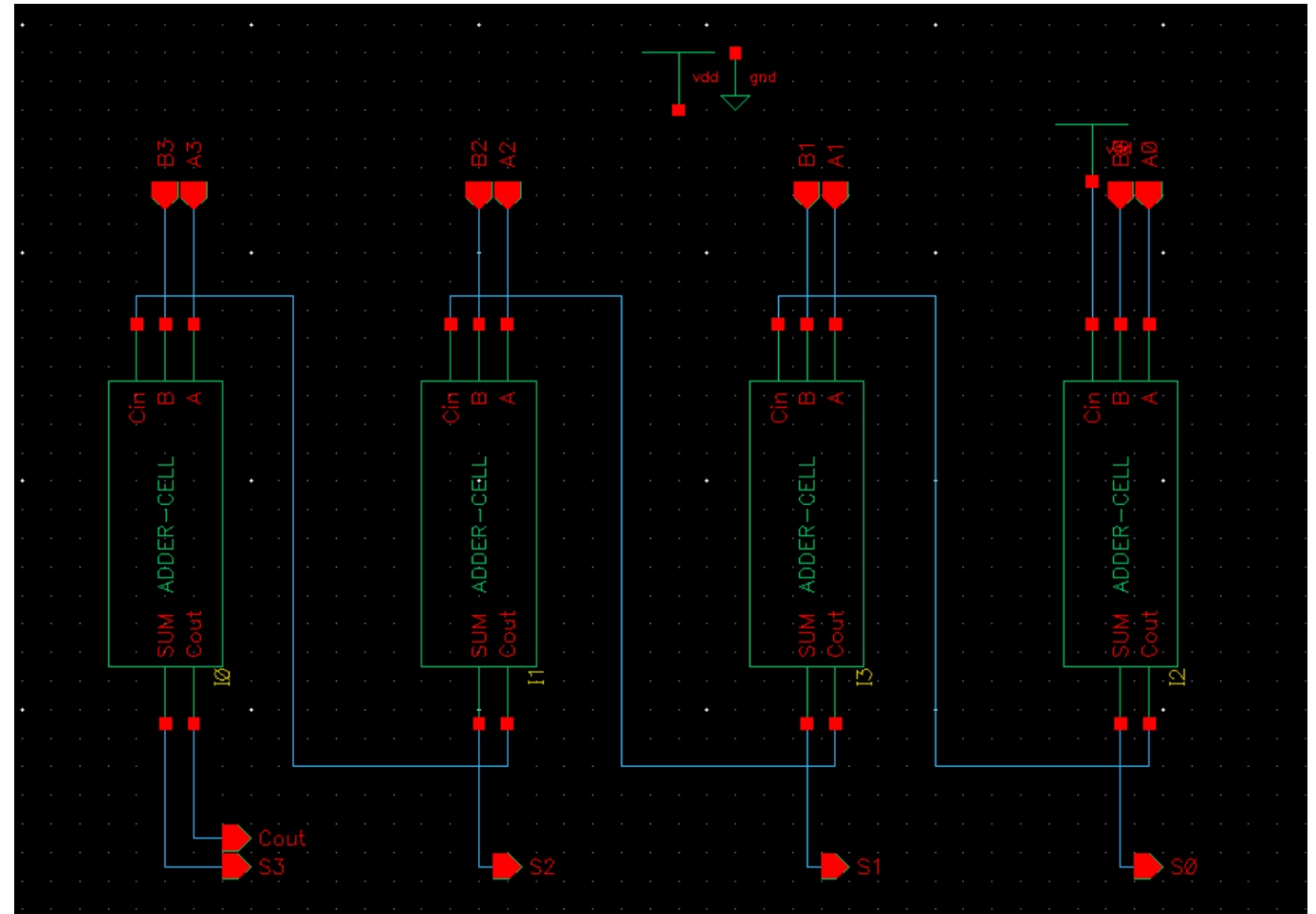


Circuit Components...

11

5-bit ADDER

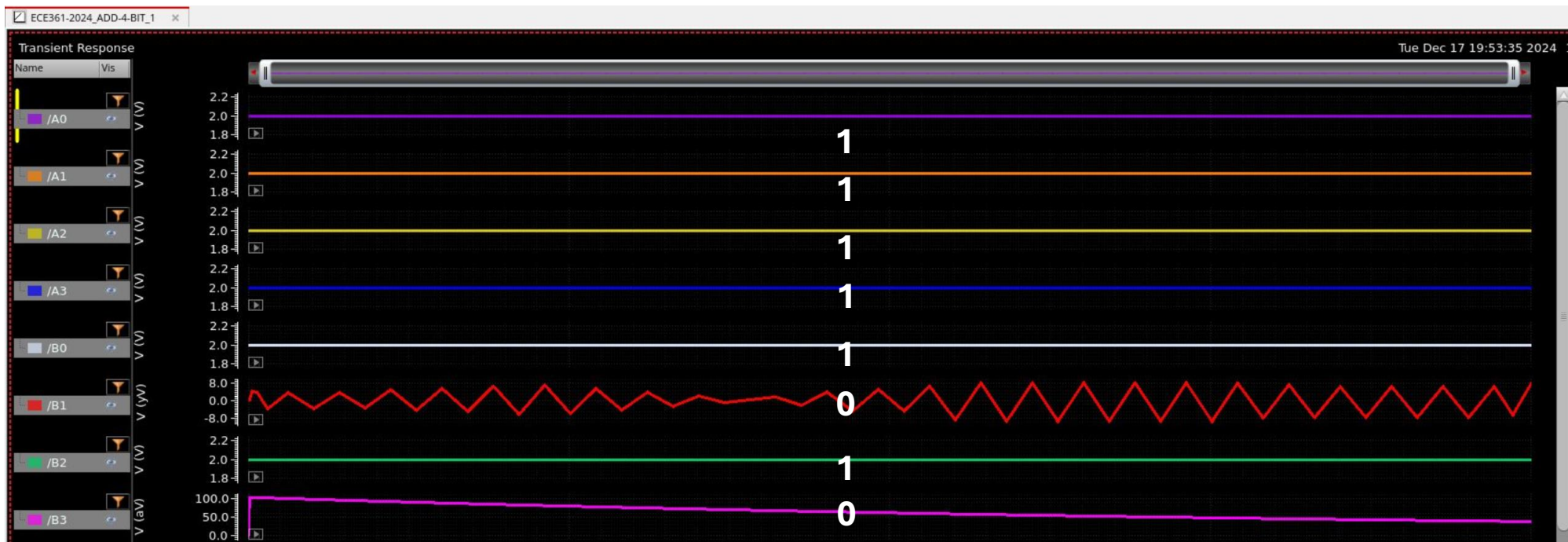
- A carry ripple adder made of 5 full adders.
- The first carry-in input is tied to VDD because this adder will only be used for 2's complement subtraction.



Circuit Components...

12

5-bit ADDER – Simulation (inputs)



*Jitters are because it is nano-Volts

Only 1 period

Just 1 addition (subtraction)

B: 0101

A: 1111

Circuit Components...

13

5-bit ADDER – Simulation (outputs)

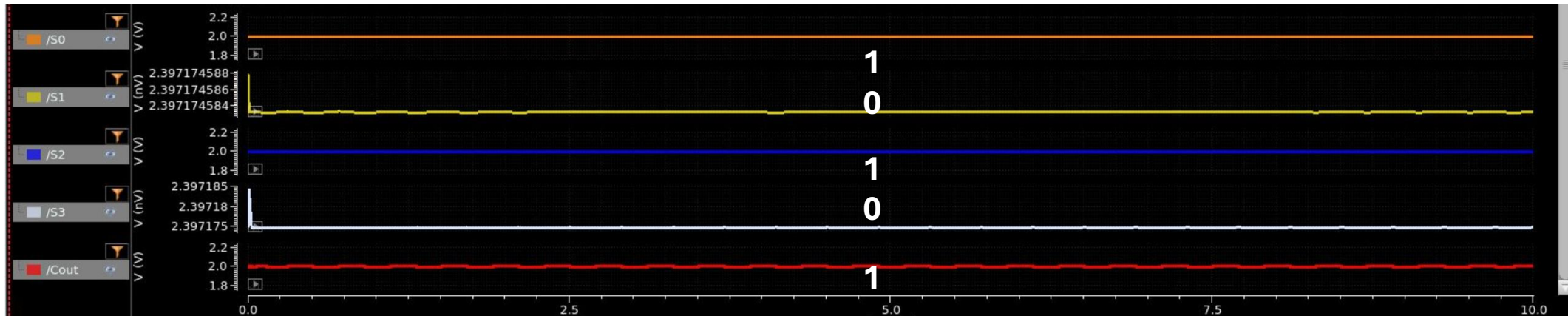
*Jitters are
because it is
nano-Volts

Only 1 period

Just 1 addition
(subtraction)

B: 0101
A: 1111
(+1)

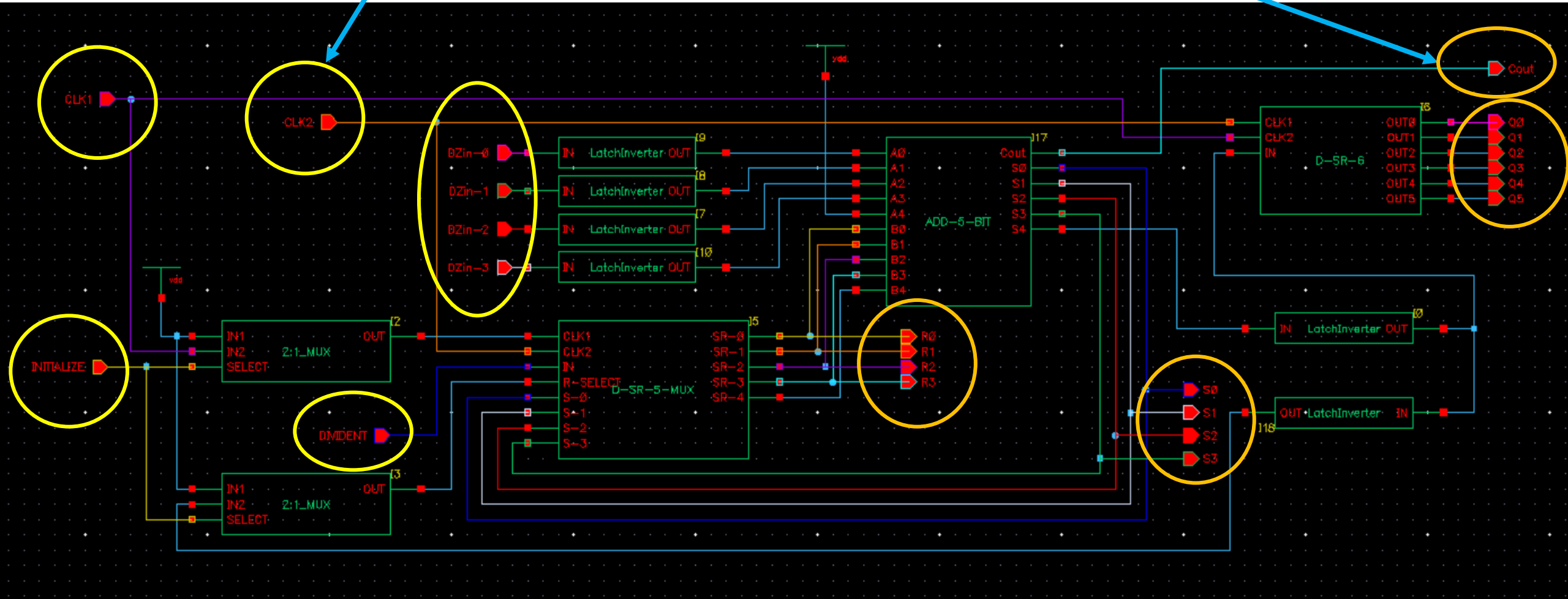
10101



Divider

Inputs

Outputs

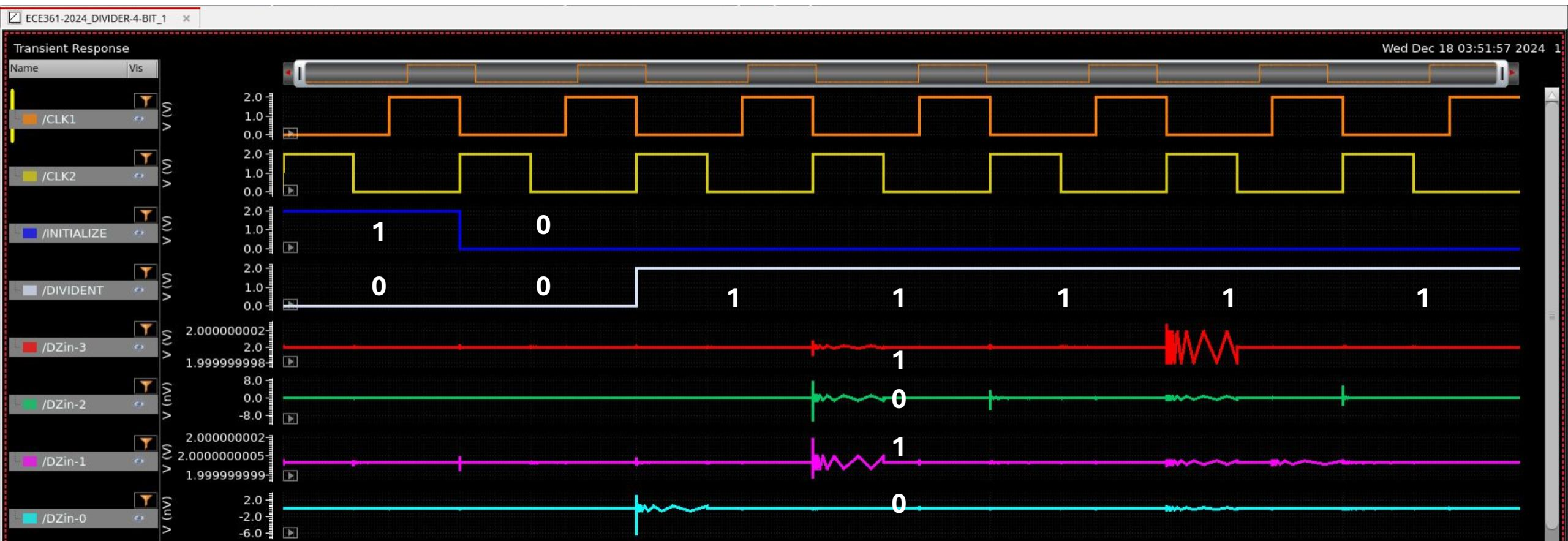


Divider - Timing

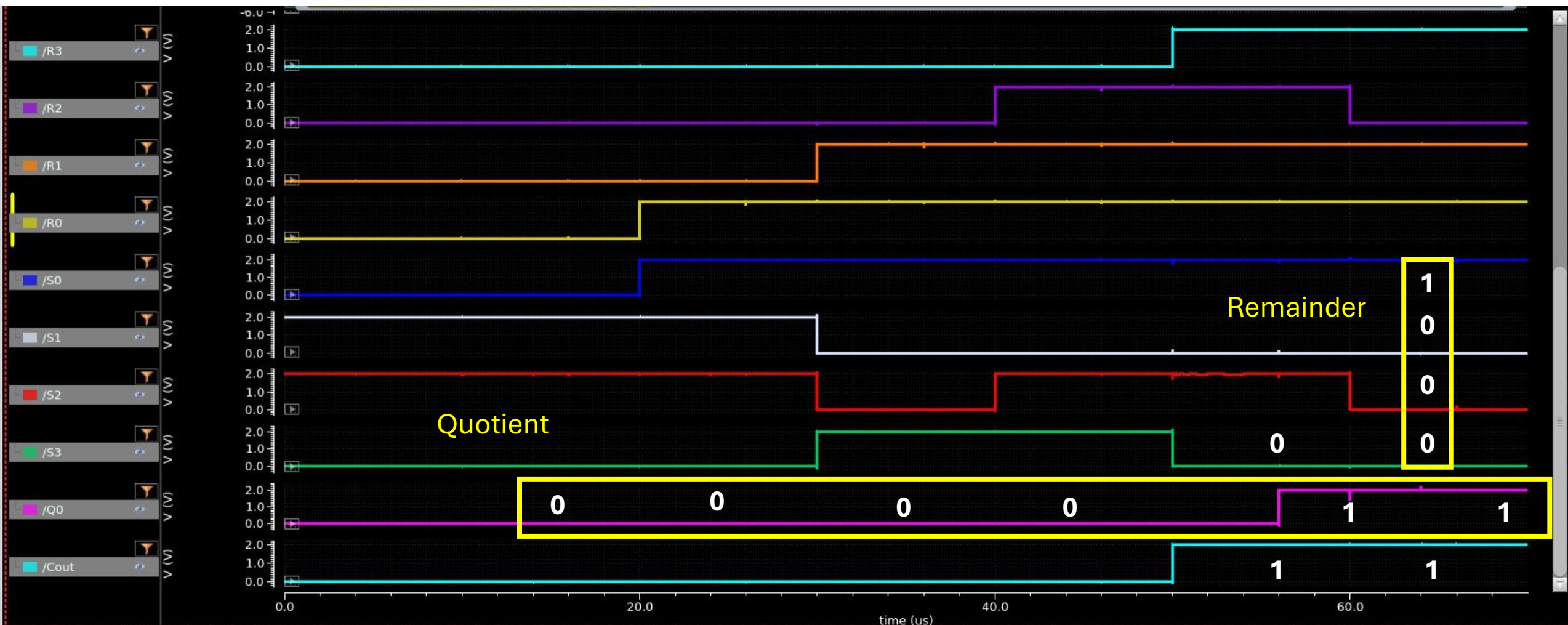
- The timing for this circuit involves two non-overlapping clocks.
- Both run on 10 microsecond periods, with 1 nanosecond rise and fall times, as well as widths of 4 microseconds.
- The first clock, CLK1, has a delay of 6us, while CLK2 has no delay.
- I chose this so there would not be a half period delay in the 5-bit MUX shift register.
 - I had some issues with the ADDER when I had CLK1 go first and there was the half period delay.
- The final output for Q0 is delayed by half a period.
 - I wired CLK2 into the CLK1 input.
 - The register was having problems when CLK1 started low and CLK2 high, so I swapped the inputs.
 - The problem was fixed, but the half period delay appeared again.

Divider – Test Case: 31/10

Divider – Test Case: 31/10... Inputs



Divider – Test Case: 31/10... Outputs

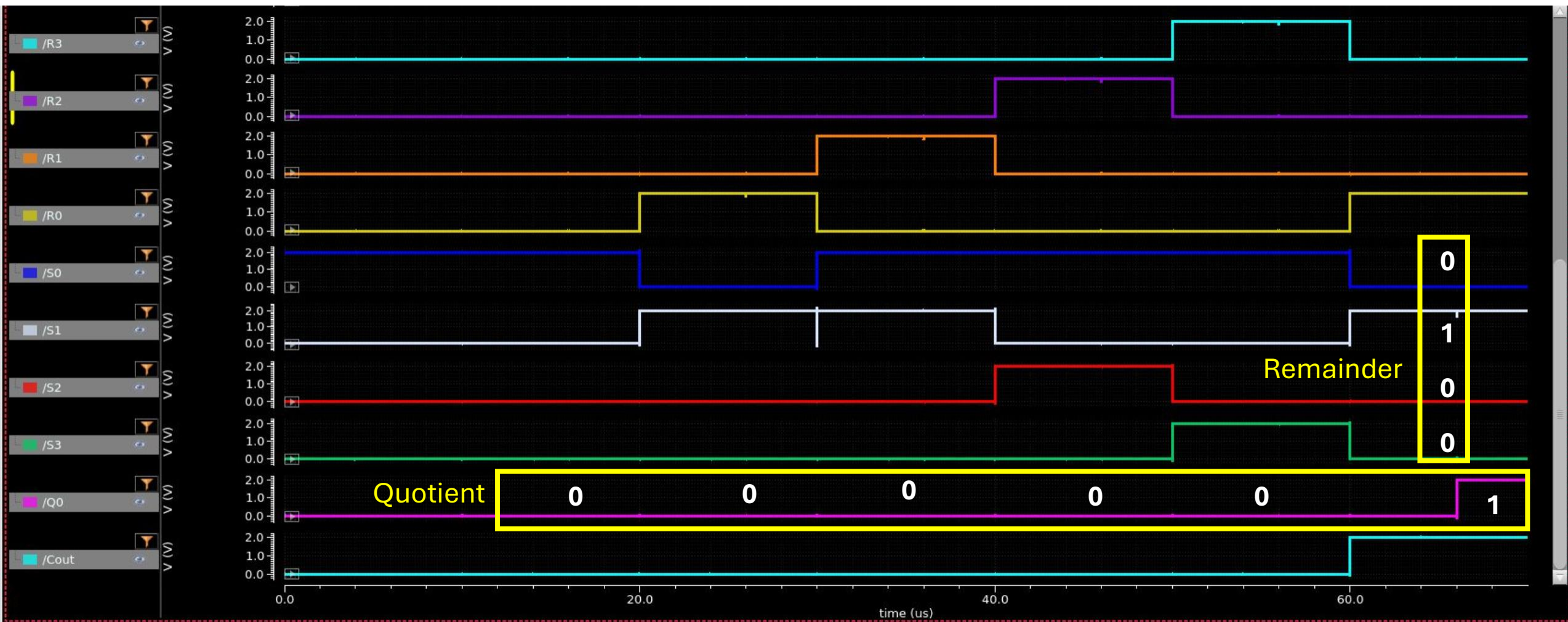


$$011111 / 01010 = 000011 \text{ R } 00001$$

Divider – Test Case: 17/15

			Cout											
DUMMY SHIFT				0	0	0	0	0	0	1	0	0	0	1
				1	0	0	0	1						
			0	1	0	0	0	1						
SHIFT 1				0	0	0	0	0	1	0	0	0	1	x
				1	0	0	0	1						
			0	1	0	0	0	1						
SHIFT 2				0	0	0	0	1	0	0	0	1	x	x
				1	0	0	0	1						
			0	1	0	0	1	0						
SHIFT 3				0	0	0	1	0	0	0	1	x	x	x
				1	0	0	0	1						
			0	1	0	0	1	1						
SHIFT 4				0	0	1	0	0	0	1	x	x	x	x
				1	0	0	0	1						
			0	1	0	1	0	1						
SHIFT 5				0	1	0	0	0	1	x	x	x	x	x
				1	0	0	0	1						
			0	0	1	0	0	1						
SHIFT 6				1	0	0	0	1	x	x	x	x	x	x
				1	0	0	0	1						
			1	0	0	0	1	0						
QUOTIENT:							1		i.e.	17/15 = 1 R2				
REMAINDER:						10								

Divider – Test Case: 17/15... Outputs

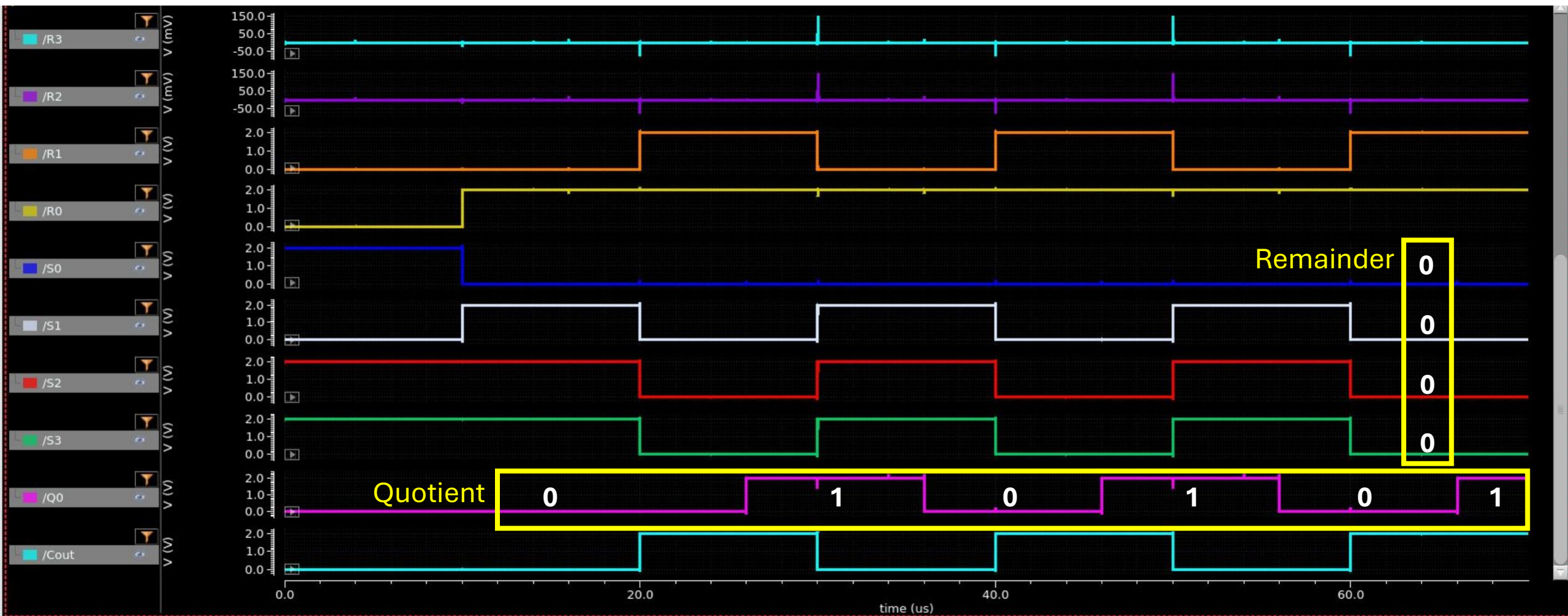


$$010001 / 01111 = 000001 R 00010$$

Divider – Test Case: 63/3

			Cout											
DUMMY SHIFT				0	0	0	0	0	1	1	1	1	1	1
				1	1	1	0	1						
			0	1	1	1	0	1						
SHIFT 1				0	0	0	0	1	1	1	1	1	1	x
				1	1	1	0	1						
			0	1	1	1	1	0						
SHIFT 2				0	0	0	1	1	1	1	1	1	x	x
				1	1	1	0	1						
			1	0	0	0	0	0						
SHIFT 3				0	0	0	0	1	1	1	1	x	x	x
				1	1	1	0	1						
			0	1	1	1	1	0						
SHIFT 4				0	0	0	1	1	1	1	x	x	x	x
				1	1	1	0	1						
			1	0	0	0	0	0						
SHIFT 5				0	0	0	0	1	1	x	x	x	x	x
				1	1	1	0	1						
			0	1	1	1	1	0						
SHIFT 6				0	0	0	1	1	x	x	x	x	x	x
				1	1	1	0	1						
			1	0	0	0	0	0						
QUOTIENT:				10101				i.e.	63/3 = 21 R0					
REMAINDER:				0										

Divider – Test Case: 63/3

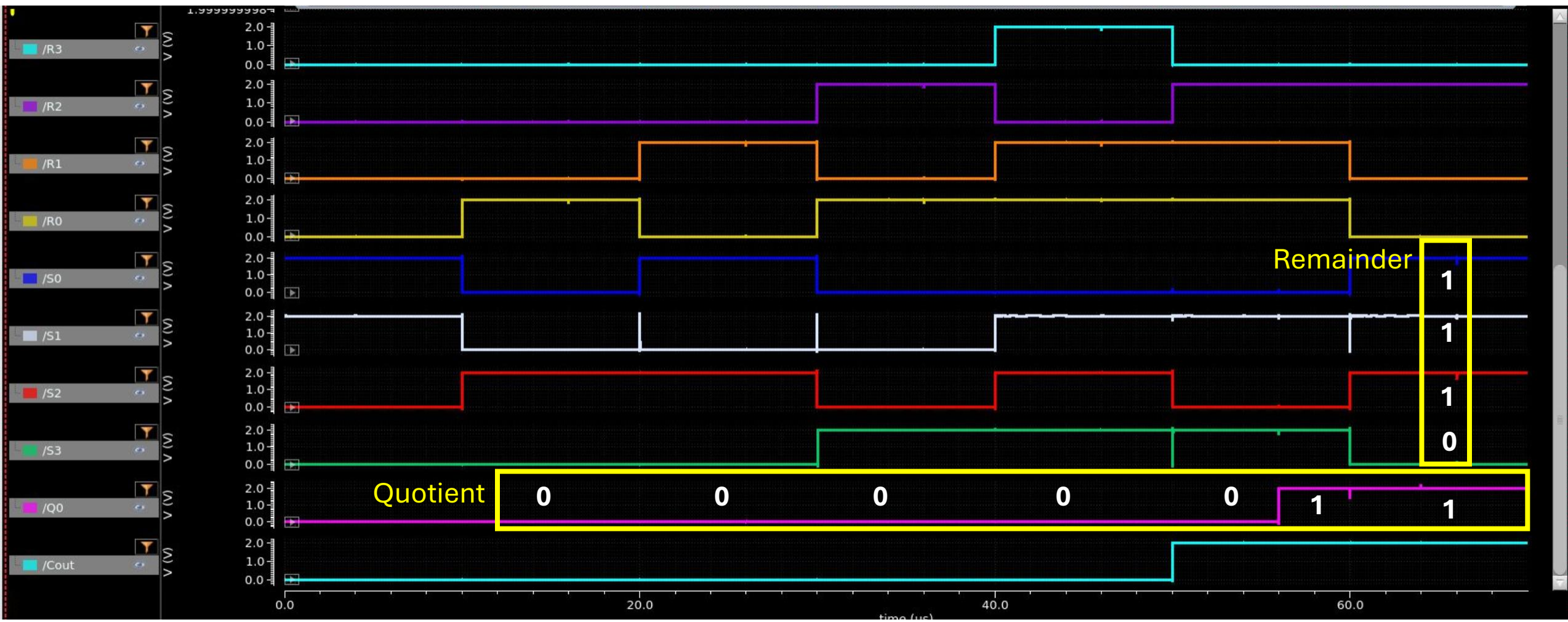


$$111111 / 00011 = 010101 \text{ R } 00000$$

Divider – Test Case: 46/13

			Cout												
DUMMY SHIFT				0	0	0	0	0		1	0	1	1	1	0
				1	0	0	1	1							
			0	1	0	0	1	1							
SHIFT 1				0	0	0	0	1		0	1	1	1	0	x
				1	0	0	1	1							
			0	1	0	1	0	0							
SHIFT 2				0	0	0	1	0		1	1	1	0	x	x
				1	0	0	1	1							
			0	1	0	1	0	1							
SHIFT 3				0	0	1	0	1		1	1	0	x	x	x
				1	0	0	1	1							
			0	1	1	0	0	0							
SHIFT 4				0	1	0	1	1		1	0	x	x	x	x
				1	0	0	1	1							
			0	1	1	1	1	0							
SHIFT 5				1	0	1	1	1		0	x	x	x	x	x
				1	0	0	1	1							
			1	0	1	0	1	0							
SHIFT 6				1	0	1	0	0		x	x	x	x	x	x
				1	0	0	1	1							
			1	0	0	1	1	1							
QUOTIENT:				1					i.e.		46/13 = 3 R7				
REMAINDER:				10											

Divider – Test Case: 46/13



$$101110 / 01101 = 000011 \text{ R } 00111$$

Thank You

Best, Colin